

Compiling VGDL into Causal Models

Mohit Jiwatode

*Institute for Information Processing
Leibniz University Hannover
Hannover, Germany
jiwatode@tnt.uni-hannover.de*

Bodo Rosenhahn

*Institute for Information Processing
Leibniz University Hannover
Hannover, Germany
rosenhahn@tnt.uni-hannover.de*

Alexander Dockhorn

*SDU Metaverse Lab
University of Southern Denmark
Odense, Denmark
adoc@sdu.dk*

Abstract—Reinforcement learning and large language models often struggle to accurately capture the causal mechanics of game environments. Standard reinforcement learning agents tend to rely on spurious correlations, while large language models are prone to hallucinating game rules. Although causal reinforcement learning improves interpretability, there is currently no formal methodology to map complex game mechanics directly into causal models. To address this, we propose a deterministic framework that compiles games specified in the Video Game Description Language into Dynamic Structural Causal Models. Rather than inferring causal structures from gameplay traces or noisy large language models’ outputs, our methodology directly translates game components, including sprite dynamics, interaction rules, and termination conditions, into explicit structural equations. Each game tick represents a causal transition from state variables at time t to $t+1$. By establishing this grounded mapping, the approach guarantees absolute causal fidelity to the ground-truth game mechanics. The resulting models offer transparent causal pathways that support counterfactual reasoning, causal reinforcement learning agent training, and procedural content validation. This framework provides a principled bridge between symbolic game descriptions and causally grounded game AI.

Index Terms—GVGAI, VGDL, Large Language Models, Causality, Causal Reinforcement Learning

I. INTRODUCTION

Reinforcement learning (RL) is widely used in modern game AI, but standard methods struggle to understand underlying causal game mechanics. The deep learning models used in RL are generally opaque, which limits the interpretability of their decisions [1]. Games frequently involve hidden rules or changing environments (e.g., new enemies or game phases) which pose adaptation challenges for naive learners [2]. To make such rules explicit to an agent, Apeldoorn et al. [3] have used hierarchical knowledge bases that are updated while playing the game. As those are learned by observation, they may include spurious correlations and therefore not represent the true rules of the underlying environment. Causal RL addresses these core problems by filtering out spurious correlations and focusing on causally relevant information, while also remaining interpretable [1]. Recent studies [4] further prove that causal explanations of RL agents significantly improve user understanding and trust.

Despite this promise, existing game benchmarks and agents rarely incorporate causality explicitly, and there exists no formal framework for mapping game mechanics into causal models.

While logic-based game description languages (e.g., GDL and systems like Ludocore [5]) have supported strategy analysis and logical forms of causal reasoning [6], they do not provide the type of structured causal models targeted here. Similarly, large language models (LLMs) applied to game reasoning suffer from hallucinations, misunderstand spatial relationships, and often perform subpar in gameplay [7], [8]. Although incorporating causal models can improve their understanding of game mechanics [9], LLMs may still misidentify such models, and without ground truth, verification is often intractable. We address this gap by introducing a formal mapping framework that equips AI agents with deterministic causal models, enabling more efficient learning through grounded causal information.

II. BACKGROUND AND RELATED WORK

A. Causality

Causality studies cause-and-effect relationships, enabling models to predict outcomes rather than just correlations [10]. Pearl’s ladder of causation defines three levels: association (statistical correlations), intervention (actively setting variables independent of their causes), and counterfactuals (reasoning about alternative outcomes to observed events) [10].

A Structural Causal Model (SCM) is a triple (U, V, F) where U is a set of exogenous variables, V a set of endogenous variables, and $F = \{f_v\}_{v \in V}$ a set of functions deriving each $v \in V$ from its parents $\text{Pa}(v) \subseteq V$ and corresponding noise variable U_v . SCMs allow for a systematic derivation of intervention effects and therefore facilitate the design of algorithms that exploit causal structures in complex domains [10]. Furthermore, a Dynamic Structural Causal Model (DSCM) extends this framework to temporal sequences by indexing variables over discrete time steps $t \in \mathcal{T}$ [10]. In a DSCM, the functions F determine the state of $v_t \in V_t$ based on its parents in current and preceding time steps, $\text{Pa}(v_t) \subseteq V_{\leq t}$, thereby capturing the causal dependencies over time [10]. An example of a DSCM is shown in Figure 1. Madumal et al. [4] use SCMs to explain RL agent behavior, generating causal explanations by analyzing counterfactuals. Hammond et al. [11] extend causal modeling to games, proposing “causal games” that encode agents’ strategies and dependencies in a unified framework. These works illustrate how causal models offer insight into decision-making and prediction beyond correlations.

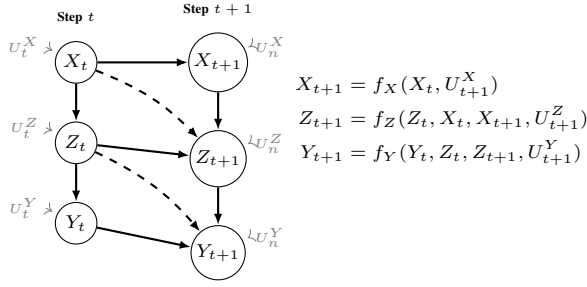


Fig. 1: Dynamic SCM: Graph (left) showing causal dependencies and structural equations (right) defining recursive assignments.

B. General Video Game AI (GVGAI) and Video Game Description Language (VGDL)

The GVGAI framework [12] allows comparing the performance of AI agents in playing a large corpus of arcade-like video games using a unified interface. Each game is defined in VGDL, a lightweight, human-readable language format consisting of a sprite set, a level mapping, a termination set, and an interaction set. While the sprite set describes all components of a game, the termination and interaction set describes the game’s logic. The latter does so by describing the outcome of collision events in terms of cause and effect relationships, effectively representing a high-level SCM.

Our approach models a video game as a *dynamic* SCM: a two-time-slice graph in which variables at time t influence those at $t + 1$. Prior work by Jiwatode et al. [9] and Chen et al. [13] integrates SCMs with LLMs for causal induction in diverse environments. We take a complementary route: rather than prompt an LLM to predict rules, we deterministically compile the known VGDL rules into an SCM. This guarantees causal fidelity (the model matches the ground-truth game mechanics) without relying on noisy language-model inference.

III. DYNAMIC CAUSAL MODEL OF GAMES

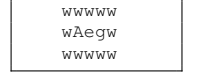
We model a VGDL game as a Dynamic SCM (DSCM), focusing on mapping VGDL directly to SCM components rather than learning correlations from traces [9]. While VGDL encodes these elements compactly, formalizing them as an SCM enables causal RL, interventions, and counterfactual analysis, which VGDL alone does not support for decision-making. Each game tick is treated as a causal transition from state t to $t + 1$, where sprite dynamics, layout, interactions, and termination rules define the causal structure.

Sprites are mapped to state representations defined by the dynamics of the selected games. We propose the following labels based on the possible descriptions of sprites in the SpriteSet in a VGDL: Position, Orientation, Velocity, IsAlive, Resource, Health, and Time. Not every selected game uses every attribute. For example, *basicgame* (see Figure 2) uses position, orientation, velocity, alive-state, live-instance counts, and terminal-state variables, while resource and health variables are needed only for games

```

BasicGame
SpriteSet
  avatar > MovingAvatar
  wall > Immovable
  goal > Immovable
  enemy > RandomNPC
InteractionSet
  avatar wall > stepBack
  goal avatar > killSprite
  avatar enemy > killSprite
LevelMapping
  A > avatar
  w > wall
  g > goal
  e > enemy
TerminationSet
  SpriteCounter stype=goal
  limit=0 win=True
  SpriteCounter stype=avatar
  limit=0 win=False

```



(b) Level layout

(a) VGDL specification

Fig. 2: Basicgame definition and example level layout

whose VGDL dynamics include collection, inventory, damage, or health-based interactions. For a game tick t , the general endogenous state can be written as

$$\mathcal{V}^t = \{P_s^t, O_s^t, Vel_s^t, A_s^t, R_s^t, H_s^t, T^t, Count_s^t, Terminal_k^t \mid s \in \mathcal{S}, k \in \mathcal{K}\}. \quad (1)$$

where $\mathcal{S}$ is the set of sprite types and instances, and $\mathcal{K}$ is the set of termination conditions. Here, P_s^t denotes the position of sprite s , O_s^t its orientation, Vel_s^t its velocity, A_s^t whether the sprite is alive or active, R_s^t its resource state, H_s^t its health state, and T^t the current game time or tick. The variable $Count_s^t$ denotes the number of live instances of a sprite type, and $Terminal_k^t$ denotes whether a win or loss condition has been reached. In Table I, these variables are abbreviated as P , O , V , A , $Count$, and $Terminal$ to keep the mapping readable.

The exogenous variables include the player action, random choices, and the initial layout:

$$\mathcal{U}^t = \{U_{\text{action}}^t, U_{\text{rng}}^t, Init_c^0\}. \quad (2)$$

The variable U_{action}^t represents the action selected by the player or agent. The variable U_{rng}^t represents randomness used by stochastic sprite classes such as *RandomNPC*. The variable $Init_c^0$ represents the level-layout symbol at initialization. The layout is exogenous because it is given before play begins and determines which sprite instances exist and where they are placed. The VGDL and level layout for *basicgame* are shown in Figure 2, and their corresponding SCM mapping is given in Table I.

The transition from t to $t + 1$ is defined by structural functions:

$$X_i^{t+1} := f_i(\text{Pa}(X_i^{t+1})), \quad (3)$$

where $\text{Pa}(X_i^{t+1})$ denotes the causal parents of X_i^{t+1} . These parents are not learned from correlations. They are obtained from the VGDL elements: *SpriteSet*, *LevelMapping*, *InteractionSet*, and *TerminationSet*. Table I gives the mapping for *basicgame*.

TABLE I: Concrete mapping for basicgame (see Figure 2). Notation: P = position, O = orientation, V = velocity, A = alive, $Count$ = live-instance count, $Terminal$ = win/loss node, U = exogenous input, RNG = random input.

VGDL element	SCM nodes	SCM edges	Structural equation
SpriteSet			
avatar > MovingAvatar	$U_{\text{action}}^t, P_{\text{avatar}}^t$ $O_{\text{avatar}}^t, V_{\text{avatar}}^t$ $A_{\text{avatar}}^t, P_{\text{avatar}}^{t+1}$ $O_{\text{avatar}}^{t+1}, V_{\text{avatar}}^{t+1}$ A_{avatar}^{t+1}	$U_{\text{action}}^t \rightarrow P_{\text{avatar}}^{t+1}, U_{\text{action}}^t \rightarrow O_{\text{avatar}}^{t+1}$ $U_{\text{action}}^t \rightarrow V_{\text{avatar}}^{t+1}, P_{\text{avatar}}^t \rightarrow P_{\text{avatar}}^{t+1}$ $O_{\text{avatar}}^t \rightarrow P_{\text{avatar}}^{t+1}, V_{\text{avatar}}^t \rightarrow P_{\text{avatar}}^{t+1}$ $A_{\text{avatar}}^t \rightarrow A_{\text{avatar}}^{t+1}$	$P_{\text{avatar}}^{t+1} := f_{\text{pos}}(P_{\text{avatar}}^t, O_{\text{avatar}}^t, V_{\text{avatar}}^t, U_{\text{action}}^t)$ $O_{\text{avatar}}^{t+1} := f_{\text{ori}}(O_{\text{avatar}}^t, U_{\text{action}}^t)$ $V_{\text{avatar}}^{t+1} := f_{\text{vel}}(V_{\text{avatar}}^t, U_{\text{action}}^t)$
wall > Immovable	$P_{\text{wall}}^t, A_{\text{wall}}^t$ $P_{\text{wall}}^{t+1}, A_{\text{wall}}^{t+1}$	$P_{\text{wall}}^t \rightarrow P_{\text{wall}}^{t+1}, A_{\text{wall}}^t \rightarrow A_{\text{wall}}^{t+1}$	$P_{\text{wall}}^{t+1} := P_{\text{wall}}^t$ $A_{\text{wall}}^{t+1} := A_{\text{wall}}^t$
goal > Immovable	$P_{\text{goal}}^t, A_{\text{goal}}^t$ $P_{\text{goal}}^{t+1}, A_{\text{goal}}^{t+1}$ $Count_{\text{goal}}^{t+1}$	$P_{\text{goal}}^t \rightarrow P_{\text{goal}}^{t+1}, A_{\text{goal}}^t \rightarrow A_{\text{goal}}^{t+1}$ $A_{\text{goal}}^{t+1} \rightarrow Count_{\text{goal}}^{t+1}$	$P_{\text{goal}}^{t+1} := P_{\text{goal}}^t$ $A_{\text{goal}}^{t+1} := A_{\text{goal}}^t$ $Count_{\text{goal}}^{t+1} := f_{\text{count}}(A_{\text{goal}}^{t+1})$
enemy > RandomNPC	$U_{\text{enemy_rng}}^t, P_{\text{enemy}}^t$ $O_{\text{enemy}}^t, V_{\text{enemy}}^t$ $A_{\text{enemy}}^t, P_{\text{enemy}}^{t+1}$ $O_{\text{enemy}}^{t+1}, V_{\text{enemy}}^{t+1}$ A_{enemy}^{t+1}	$U_{\text{enemy_rng}}^t \rightarrow P_{\text{enemy}}^{t+1}, U_{\text{enemy_rng}}^t \rightarrow O_{\text{enemy}}^{t+1}$ $U_{\text{enemy_rng}}^t \rightarrow V_{\text{enemy}}^{t+1}, P_{\text{enemy}}^t \rightarrow P_{\text{enemy}}^{t+1}$ $O_{\text{enemy}}^t \rightarrow P_{\text{enemy}}^{t+1}, V_{\text{enemy}}^t \rightarrow P_{\text{enemy}}^{t+1}$ $A_{\text{enemy}}^t \rightarrow A_{\text{enemy}}^{t+1}$	$P_{\text{enemy}}^{t+1} := f_{\text{pos}}(P_{\text{enemy}}^t, O_{\text{enemy}}^t, V_{\text{enemy}}^t, U_{\text{enemy_rng}}^t)$ $O_{\text{enemy}}^{t+1} := f_{\text{ori}}(O_{\text{enemy}}^t, U_{\text{enemy_rng}}^t)$ $V_{\text{enemy}}^{t+1} := f_{\text{vel}}(V_{\text{enemy}}^t, U_{\text{enemy_rng}}^t)$
LevelMapping + layout			
LevelMapping + layout	$Init_c^0, P_s^t$ $A_s^t, c \mapsto s$	$Init_c^0 \rightarrow P_s^t, Init_c^0 \rightarrow A_s^t$	$P_s^t := \text{init_position}(Init_c^0)$ $A_s^t := \text{init_existence}(Init_c^0)$ for any layout symbol c mapped to sprite s
InteractionSet			
avatar wall > stepBack	$U_{\text{action}}^t, P_{\text{avatar}}^t$ $P_{\text{wall}}^t, P_{\text{avatar}}^{t+1}$	$U_{\text{action}}^t \rightarrow P_{\text{avatar}}^{t+1}, P_{\text{avatar}}^t \rightarrow P_{\text{avatar}}^{t+1}$ $P_{\text{wall}}^t \rightarrow P_{\text{avatar}}^{t+1}$	$P_{\text{avatar}}^{t+1} := \text{stepBack}(P_{\text{avatar}}^t, P_{\text{wall}}^t, U_{\text{action}}^t)$
goal avatar > killSprite	$P_{\text{goal}}^t, P_{\text{avatar}}^t$ $A_{\text{goal}}^t, A_{\text{avatar}}^t$ A_{goal}^{t+1}	$P_{\text{goal}}^t \rightarrow A_{\text{goal}}^{t+1}, P_{\text{avatar}}^t \rightarrow A_{\text{goal}}^{t+1}$ $A_{\text{goal}}^t \rightarrow A_{\text{goal}}^{t+1}, A_{\text{avatar}}^t \rightarrow A_{\text{goal}}^{t+1}$	$A_{\text{goal}}^{t+1} := \text{killSprite}(A_{\text{goal}}^t, \text{overlap}(P_{\text{goal}}^t, P_{\text{avatar}}^t), A_{\text{avatar}}^t)$
avatar enemy > killSprite	$P_{\text{avatar}}^t, P_{\text{enemy}}^t$ $A_{\text{avatar}}^t, A_{\text{enemy}}^t$ A_{avatar}^{t+1}	$P_{\text{avatar}}^t \rightarrow A_{\text{avatar}}^{t+1}, P_{\text{enemy}}^t \rightarrow A_{\text{avatar}}^{t+1}$ $A_{\text{avatar}}^t \rightarrow A_{\text{avatar}}^{t+1}, A_{\text{enemy}}^t \rightarrow A_{\text{avatar}}^{t+1}$	$A_{\text{avatar}}^{t+1} := \text{killSprite}(A_{\text{avatar}}^t, \text{overlap}(P_{\text{avatar}}^t, P_{\text{enemy}}^t), A_{\text{enemy}}^t)$
TerminationSet			
SpriteCounter stye=goal limit=0 win=True	$A_{\text{goal}}^{t+1}, Count_{\text{goal}}^{t+1}$ $Terminal_{\text{goal}}^{t+1}$	$A_{\text{goal}}^{t+1} \rightarrow Count_{\text{goal}}^{t+1}$ $Count_{\text{goal}}^{t+1} \rightarrow Terminal_{\text{goal}}^{t+1}$	$Count_{\text{goal}}^{t+1} := f_{\text{count}}(A_{\text{goal}}^{t+1})$ $Terminal_{\text{goal}}^{t+1} := \mathbf{1}[Count_{\text{goal}}^{t+1} \leq 0]$
SpriteCounter stye=avatar limit=0 win=False	$A_{\text{avatar}}^{t+1}, Count_{\text{avatar}}^{t+1}$ $Terminal_{\text{avatar}}^{t+1}$	$A_{\text{avatar}}^{t+1} \rightarrow Count_{\text{avatar}}^{t+1}$ $Count_{\text{avatar}}^{t+1} \rightarrow Terminal_{\text{avatar}}^{t+1}$	$Count_{\text{avatar}}^{t+1} := f_{\text{count}}(A_{\text{avatar}}^{t+1})$ $Terminal_{\text{avatar}}^{t+1} := \mathbf{1}[Count_{\text{avatar}}^{t+1} \leq 0]$

A. Sprite Dynamics

Each sprite class contributes a local causal mechanism. These mechanisms are derived from the sprite definitions in the `SpriteSet` of the VGDL description shown in Figure 2. In the mapping in Table I, sprite dynamics define how attributes such as position, orientation, velocity, and alive-state are updated from time t to time $t+1$.

For a moving sprite s , the next position is caused by its current position, orientation, velocity, and the relevant input:

$$P_s^{t+1} := f_{\text{pos}}(P_s^t, O_s^t, V_{\text{el}}^t, U_s^t). \quad (4)$$

For the avatar, $U_s^t = U_{\text{action}}^t$. For a random enemy, $U_s^t = U_{\text{enemy_rng}}^t$. The orientation and velocity updates are

$$O_s^{t+1} := f_{\text{ori}}(O_s^t, U_s^t), \quad V_{\text{el}}^{t+1} := f_{\text{vel}}(V_{\text{el}}^t, U_s^t). \quad (5)$$

For immovable sprites such as walls and goals, the position mechanism is an identity function: $P_s^{t+1} := P_s^t$. Their alive-state persists unless an interaction rule changes it: $A_s^{t+1} := A_s^t$.

This distinction makes the causal role of sprite classes explicit. A `MovingAvatar` has an action-dependent movement mechanism, a `RandomNPC` has an RNG-dependent movement mechanism, and an `Immovable` sprite has a persistence mechanism. The sprite dynamics, therefore, define the baseline state transition before interaction rules such as `stepBack` or `killSprite` modify the next state.

B. Interaction Mechanisms

Interaction mechanisms are derived from the `InteractionSet` in the VGDL description. They define how the state of one sprite changes when it spatially interacts with another sprite. In the mapping in Table I,

these mechanisms include rules such as `stepBack` and `killSprite`. For two sprites i and j , an interaction is activated by a spatial condition:

$$C_{ij}^t := \text{overlap}(P_i^t, P_j^t). \quad (6)$$

The resulting state update can be written as

$$X_i^{t+1} := f_{\text{int}}(X_i^t, P_i^t, P_j^t, A_i^t, A_j^t, C_{ij}^t), \quad (7)$$

where X_i^{t+1} is the affected next-state variable.

For example, the rule `avatar wall > stepBack` changes the avatar's next position when the attempted movement causes contact with a wall:

$$P_{\text{avatar}}^{t+1} := \text{stepBack}(P_{\text{avatar}}^t, P_{\text{wall}}^t, U_{\text{action}}^t). \quad (8)$$

Similarly, `killSprite` changes the alive-state of the affected sprite when the required overlap condition holds:

$$A_i^{t+1} := \text{killSprite}(A_i^t, C_{ij}^t, A_j^t). \quad (9)$$

Thus, interaction mechanisms specify causal effects of spatial contact. They do not merely record that two sprites are close to each other. They define which sprite state is changed, under which spatial condition, and by which VGDL rule.

C. Termination Mechanisms

Termination mechanisms are derived from the `TerminationSet` in the VGDL description. They define when the game reaches a win or loss state. In the mapping in Table I for `basicgame`, termination depends on live-instance counts for sprites such as the goal and the avatar.

For a sprite type s , the live-instance count is computed from the alive-state variables:

$$\text{Count}_s^{t+1} := f_{\text{count}}(A_s^{t+1}). \quad (10)$$

A terminal variable is then computed from this count:

$$\text{Terminal}_s^{t+1} := \mathbf{1}[\text{Count}_s^{t+1} \leq \text{limit}_s]. \quad (11)$$

For example, in `basicgame`, the game is

- won when no goal sprites remain:
 $\text{Terminal}_{\text{goal}}^{t+1} := \mathbf{1}[\text{Count}_{\text{goal}}^{t+1} \leq 0].$
- lost when no avatar sprites remain:
 $\text{Terminal}_{\text{avatar}}^{t+1} := \mathbf{1}[\text{Count}_{\text{avatar}}^{t+1} \leq 0].$

Termination nodes are, therefore, leaf nodes in the SCM. They summarize the consequences of earlier sprite dynamics and interaction mechanisms, but they do not directly cause movement or collision outcomes within the same transition.

IV. DISCUSSION AND FUTURE WORK

This dynamic SCM framework has several benefits. It provides *explainability* by making causal pathways explicit, allows *simulation of counterfactuals*, and supports *causal RL* by supplying a full state model to agents. It also enables *procedural content validation* by testing whether a newly generated game description is logically consistent through its implied causal model, as done by Jiwatode et al. [9] for VGDL synthesis.

One of the primary limitations of this approach is that the causal model's size scales with the complexity of the VGDL description. Additionally, games with intricate physics or hidden states may require structural approximations. Furthermore, if the actual game implementation deviates from the VGDL design (e.g., due to bugs or unmodeled nuances), errors will naturally arise in the causal mapping.

Future work should address these issues by developing semi-automated tools to extract SCMs directly from game code and by extending the framework to multi-agent or real-time games. This framework could also be extended to other games in the GVGAI suite and to other game definitions. Future research should also empirically evaluate these SCMs to quantify their impact on agent performance. Different methods to encode spatial relationships in the causal model could also be explored.

In conclusion, by mapping video games to a dynamic causal framework, we enable reasoning about actions and outcomes in games, complementing statistical and learning-based methods.

Acknowledgements:

This work is jointly supported by the “HybrInt - Hybrid Intelligence through Interpretable AI in Machine Perception and Interaction” project (Zukunft Nds, Niedersächsisches Ministerium für Wissenschaft, Grant ID: ZN4219) and the Novo Nordisk Foundation grant NNF25OC0105856.

The editing process was supported by Google's Gemini.

REFERENCES

- [1] Z. Deng, J. Jiang, G. Long, and C. Zhang, “Causal reinforcement learning: A survey,” *arXiv preprint arXiv:2307.01452*, 2023.
- [2] M. Li, J. Zhang, and E. Bareinboim, “Confounding robust deep reinforcement learning: A causal approach,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 138 292–138 325, 2026.
- [3] D. Apeldoorn and A. Dockhorn, “Exception-tolerant hierarchical knowledge bases for forward model learning,” *IEEE Transactions on Games*, p. 1–14, 2021.
- [4] P. Madumal, T. Miller, L. Sonenberg, and F. Vetere, “Explainable reinforcement learning through a causal lens,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 03, 2020, pp. 2493–2500.
- [5] A. M. Smith, M. J. Nelson, and M. Mateas, “Ludocore: A logical game engine for modeling videogames,” in *Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games*. IEEE, 2010, pp. 91–98.
- [6] M. Genesereth and M. Thielscher, “Analyzing games with logic,” in *General Game Playing*. Springer, 2022, pp. 129–149.
- [7] Y. Li, C. Lin, M. U. Nasir, P. Bontrager, J. Liu, and J. Togelius, “Gvgai-llm: Evaluating large language model agents with infinite games,” *arXiv:2508.08501*, 2025.
- [8] M. Jiwatode, R. Fuchs, R. Schmöcker, B. Rosenhahn, and A. Dockhorn, “Spatial reasoning in llm game agents: Impact of causal context and multi-step planning,” in *2026 IEEE Conference on Games (CoG)*. IEEE, 2026, pp. 1–8.
- [9] M. Jiwatode, A. Dockhorn, and B. Rosenhahn, “From gameplay traces to game mechanics: Causal induction with large language models,” *arXiv:2602.00190*, 2026.
- [10] J. Pearl, *Causality*. Cambridge University Press, 2009.
- [11] L. Hammond, J. Fox, T. Everitt, R. Carey, A. Abate, and M. Wooldridge, “Reasoning about causality in games,” *Artificial Intelligence*, vol. 320, p. 103919, 2023.
- [12] D. Perez-Liebana, S. M. Lucas, R. D. Gaina, J. Togelius, A. Khalifa, and J. Liu, *General Video Game Artificial Intelligence*. Morgan & Claypool Publishers, 2019, vol. 3, no. 2, <https://gaigresearch.github.io/gvgaibook/>.
- [13] W. Chen, J. Zhang, H. Zhu, B. Xu, Z. Hao, K. Zhang, J. Ye, and R. Cai, “Causal-aware large language models: Enhancing decision-making through learning, adapting and acting,” *arXiv:2505.24710*, 2025.